

Virtualizacija



Sadržaj



- Što je virtualizacija?
- Virtualni stroj
- Kontejneri
- Docker
- Python environments
- Usporedba

Virtualizacija



- Virtualizacija je način stvaranja virtualnih računalnih resursa:
 - Poslužitelja (*engl. Server virtualization*)
 - Mreže (*engl. Network virtualization*)
 - Pohrana podataka (*engl. Storage virtualization*)
 - Podaci (*engl. Data virtualization*)
 - Radna površina (*engl. Desktop virtualization*)
- Način rada:
 - Softverski sloj (hipervizor) radi apstrakciju hardvera računala te distribuira resurse na više virtualnih strojeva koje nazivamo gostima

Virtualni stroj (*engl. Virtual machine*)

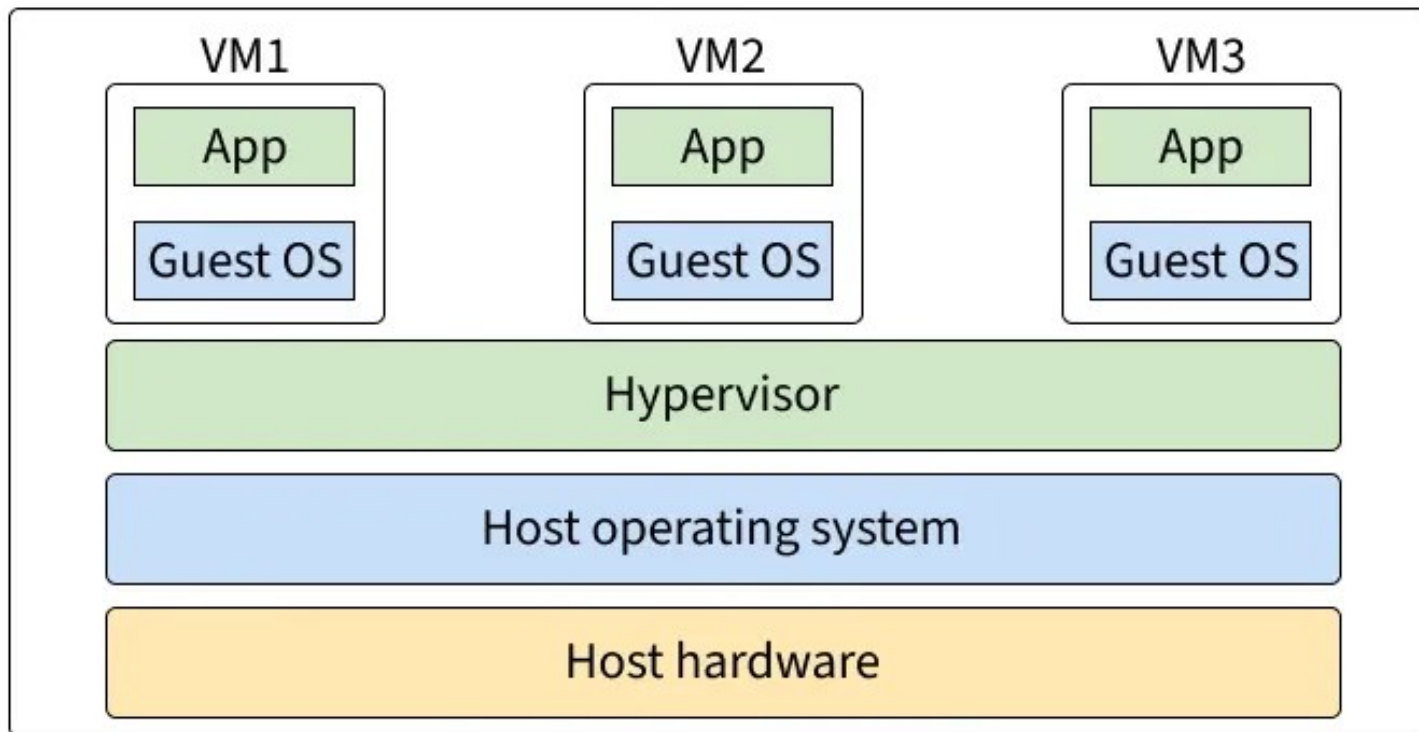


- Simuliranje fizičkog računala na operativnom sustavu poslužitelja
- Gostujući operativni sustav je u potpunosti izoliran od poslužitelja
- VM aplikacija odlučuje o dodjeli resursa (procesor, memorija, diskovi)
- Popularne aplikacije za virtualizaciju:
 - Oracle VirtualBox, QEMU (All)
 - Parallels Desktop (MacOS)
 - Hyper-V (Windows)

Virtualni stroj (*engl. Virtual machine*)



Virtual machines



Kontejneri

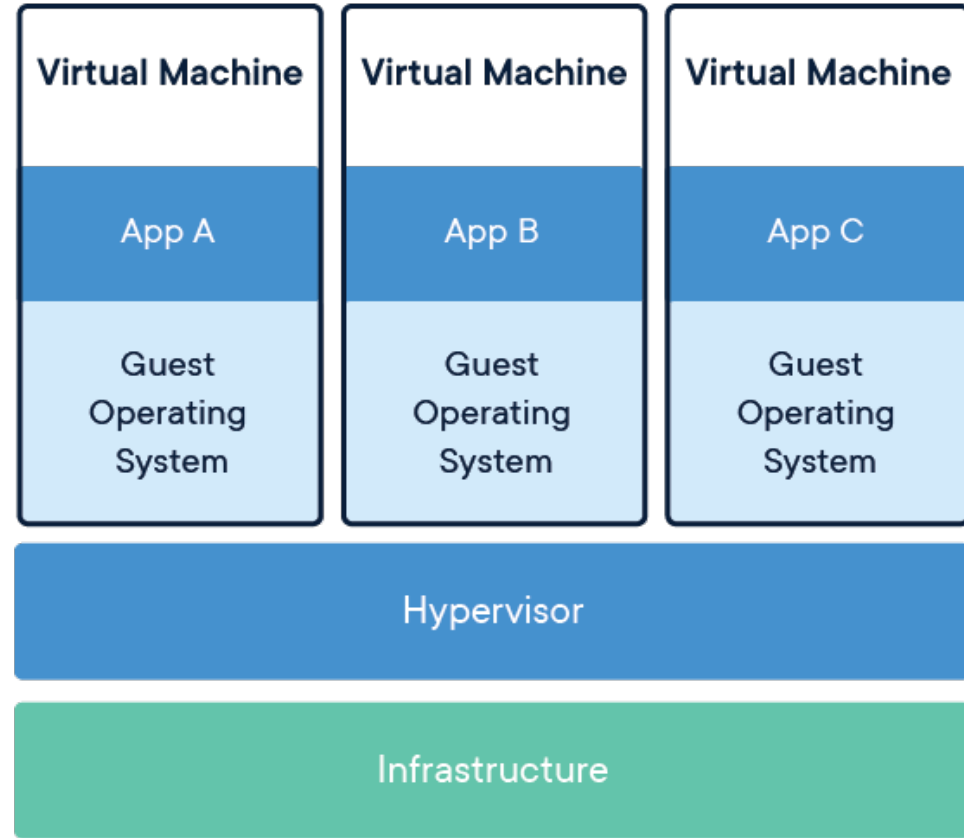
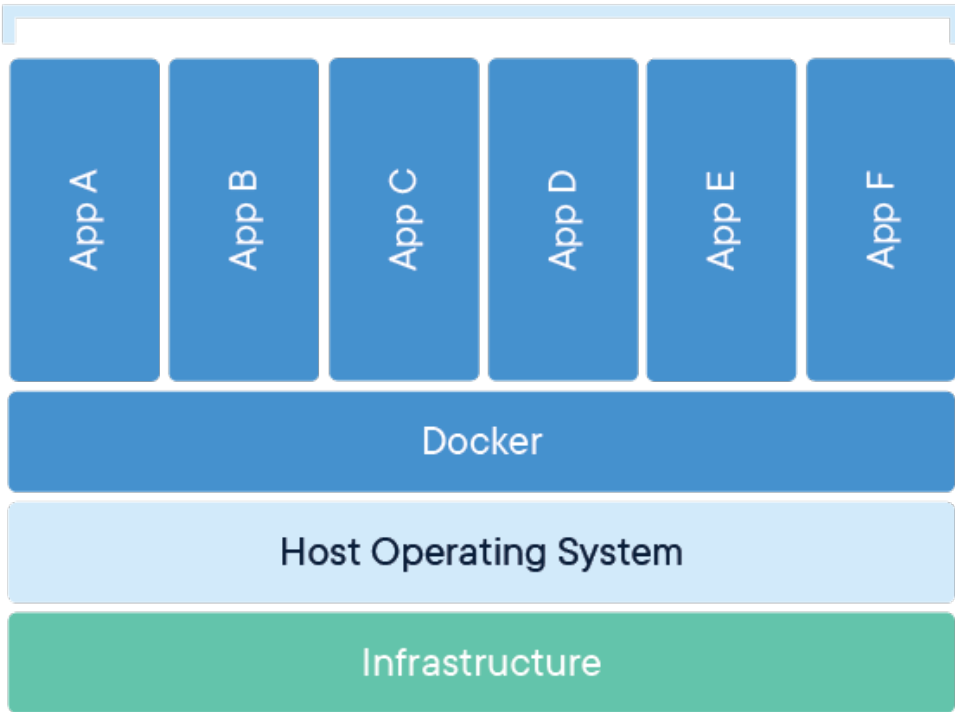


- Kontejneri su male prenosive jedinice koje dijele operativni sustav poslužitelja pri čemu im je aplikacijsko okruženje u potpunosti izolirano
- Koriste se u modernoj arhitekturi i aplikacijama u oblaku
- Bitna razlika prema VM:
 - Kontejneri dijele OS (pokreću se na kontejnerskom stroju, dok se VM vrti na hipervizoru)
 - Veličina i brzina – kontejneri su manji i brže se pokreću (cold start)
 - Izolacija i sigurnost – VM pružaju hardversku izolaciju
 - Prenosivost – kontejneri pružaju veliku fleksibilnost

Kontejneri



Containerized Applications

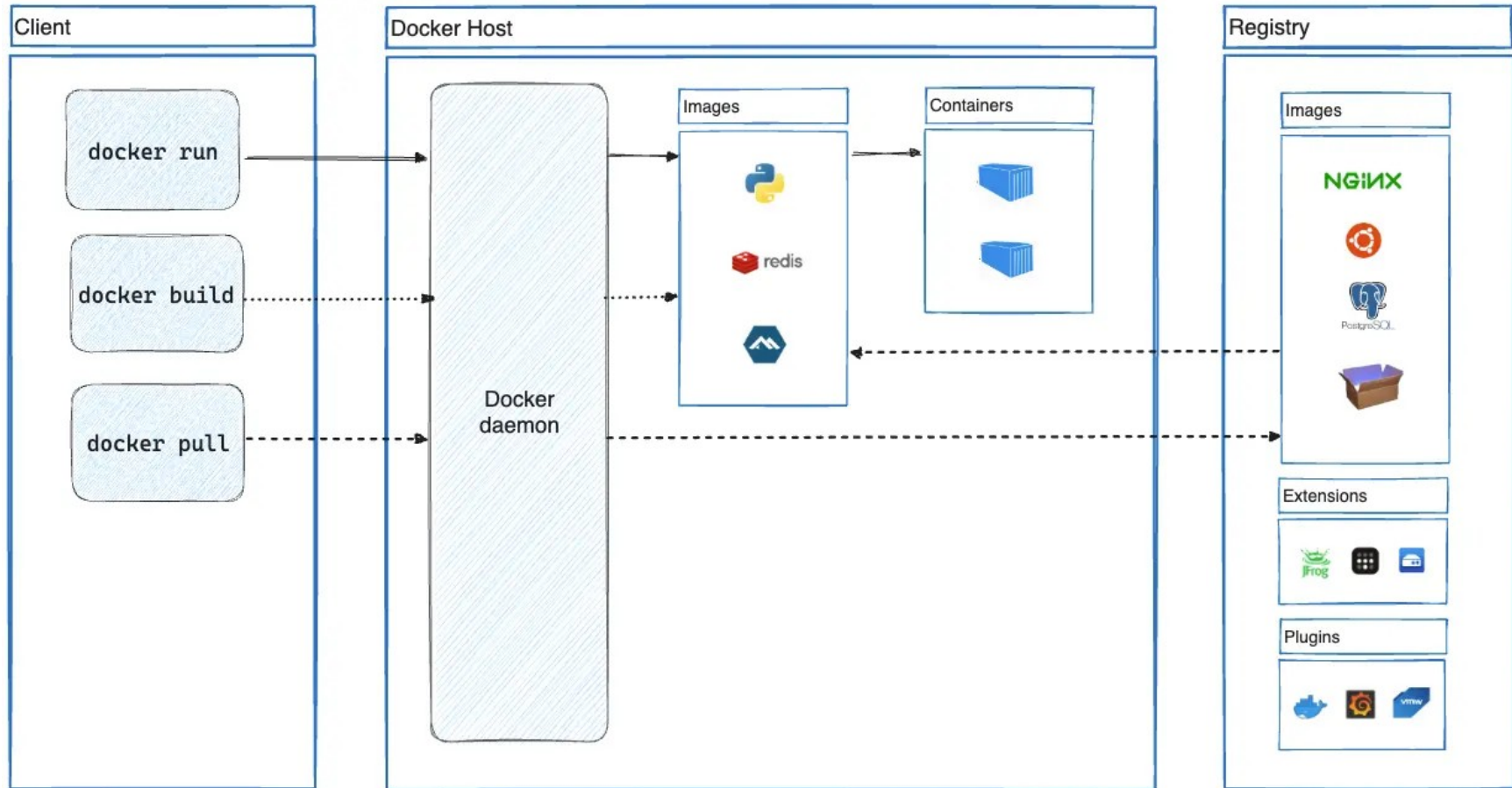


Docker



- Inicijalno razvijen kao sustav za definiciju i pokretanje kontejnera
- Danas je to platforma za izradu, dijeljenje i pokretanje aplikacija
- Docker Hub
- Docker Desktop
- Docker vs K8 (Kubernetes)

Docker



Python virtualno okruženje

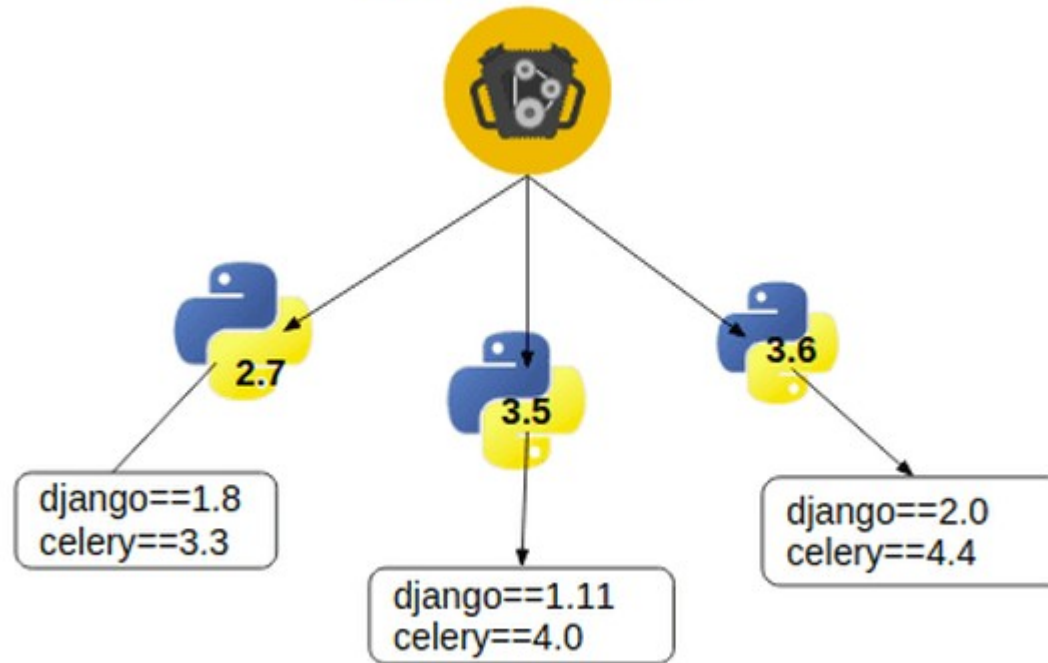


- Virtualno okruženje u ovom slučaju je izolirano okruženje za Python aplikaciju
- Nema izolacije aplikacijskog sloja, nema izolacije hardvera
- Omogućuje definiranje verzije paketa zavisnih biblioteka po projektu (odvaja sistemske pakete od projektnih)
- Koristi python verziju pomoću koje se kreira okruženje
 - **`python -m venv <naziv_direktorija>`**

Python virtualno okruženje



Virtualenv



Python virtualno okruženje vs Conda env



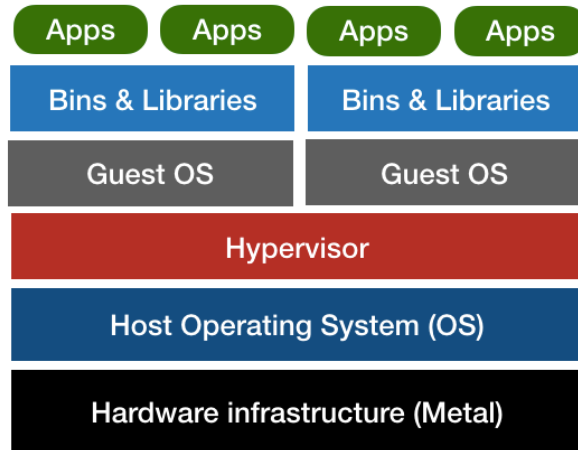
The big picture differences

Dimension	Python venv	Conda env
Scope	Python packages only	Python + non-Python libs/binaries
Resolver	pip (Python Package Index)	Conda solver (Conda/mamba channels)
Availability	Built into Python	Requires Miniconda/Anaconda/mamba
Speed	Fast to create. Installs depend on wheels/source build	Env solve/install can be slower (mamba is very fast)
Binary compatibility	Relies on wheels or local compilers	Ships prebuilt binaries for many platforms.
Disk usage	Typically small	Can be heavier (multiple copies of libs)
Reproducibility	Good with pinned versions/lock files	Very good with explicit specs (environment.yml)
CUDA/GPU	Via PyPI wheels that match system CUDA or bundled runtimes	Excellent; can install PyTorch and, CUDA toolkit in one environment.
Multi-language	Python only	Python, R, and more in one env
Portability	Easy to create on machines with Python	Easy if using the same OS/arch and channels

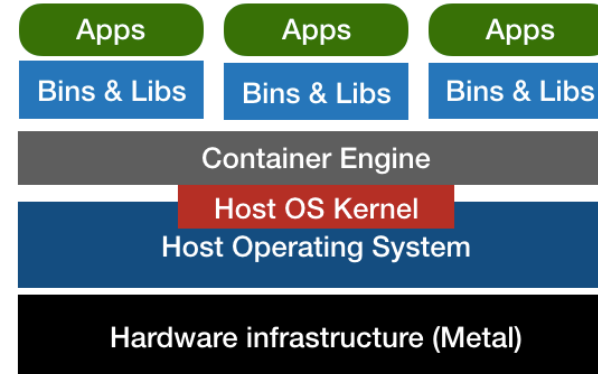
Usporedba



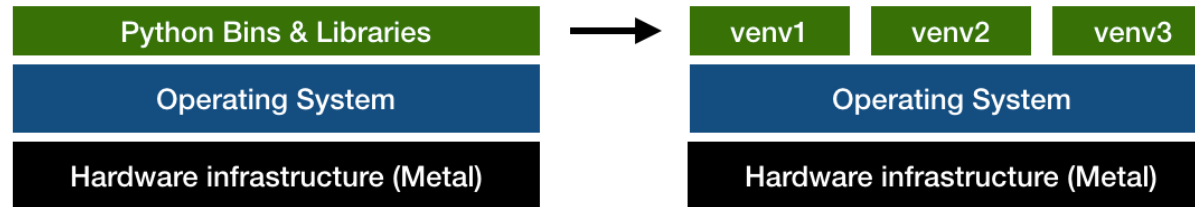
Virtual Machine



Docker Container



Virtual Environments



Usporedba (primjer)



- Koristiti značajke koje su specifične za svaki OS → VM
- Koristiti različite verzije Pythona na istom računalu → Docker
- Koristiti različite verzije zavisnih biblioteka → Python venv